

AI Upskilling Platform

Sample Lesson Validation — 5 Lessons Across Types

CONTENTS



Reading

Java Mastery: Introduction & Core Concepts

Generated 2026-05-29 · Powered by Claude claude-sonnet-4-6

Java Mastery: Introduction & Core Concepts

Topic: Java Mastery

Overview

Java solves the platform independence problem through a two-stage execution model built on bytecode and the Java Virtual Machine, captured by the slogan "write once, run anywhere." The `javac` compiler translates human-readable `.java` source files into `.class` files containing platform-neutral bytecode rather than native machine code. At execution time, the JVM (Java Virtual Machine) loads that bytecode and either interprets it or, for frequently executed methods, JIT-compiles it into optimised native instructions for the host operating system and CPU architecture. This indirection is precisely why Java dominates enterprise software: the same `.jar` artifact runs unchanged on Linux servers, Windows desktops, and Android phones. Enterprise systems such as Spring Boot services, Android applications, and core banking platforms further rely on Java's strong static typing, automatic garbage collection, and an exceptionally rich library ecosystem to build large, long-lived, maintainable codebases.

The Laws of Cricket are published by the MCC and govern every match worldwide — the same LBW law applies whether the match is at Lord's or Eden Gardens, played by India or England. A ball bowled by Bumrah in the IPL follows exactly the same rules as one bowled in a village match in Chennai. The umpire, whether standing at the MCG or Headingley, enforces the same Laws regardless of the local conditions. Java bytecode is the Laws of Cricket — written once, universally applicable. The JVM is the umpire who enforces those Laws on whatever physical ground (operating system) the match is played. Your `.java` source code is the team's game plan; `javac` compiles it into the universal `.class` bytecode format, which any JVM-certified umpire can interpret and execute. Just as the same LBW law produces different on-field rulings based on pitch conditions like bounce and pace, the JVM adapts bytecode execution to local hardware (x86 vs ARM) while keeping the same program semantics. This is why Java achieved cross-platform portability before it was common — the JVM abstraction layer absorbs all platform differences so the program never has to change, the same way a cricket team doesn't rewrite their game plan for each new ground.

Core Concepts

Encapsulation

Encapsulation bundles data (fields) and the methods that operate on that data into a single unit (a class), and restricts direct access to the fields from outside code. Fields are declared private, and

access is controlled through public getter and setter methods or purpose-built methods that express the object's behaviour. This prevents external code from putting an object into an invalid state — a `BankAccount` cannot have a negative balance if the `withdraw()` method validates the requested amount before modifying the internal balance field. Encapsulation also enables internal implementation changes without breaking external callers, since those callers interact only with the stable public API and never with the internal fields, which keeps the class free to evolve its storage and logic independently.

Think about how a cricket team's dressing room works. During a match, the batting team's strategy, the running commentary on the opposition's weaknesses, and the coach's tactical adjustments all happen entirely inside the dressing room, behind closed doors. The opposition can observe the runs scored and wickets fallen on the scoreboard (the public interface), but they cannot walk into the dressing room and inspect the captain's notes or the video analyst's breakdown of their own bowling (the private fields). In Java, the dressing room is the class boundary, and the captain's notes are the private fields. Encapsulation means the `Cricket` class declares `private int runsScored` and `private int wicketsLost`, so external classes can't directly set `runsScored = -10`, an invalid state like a team having negative runs. The only way to update runs is through the public `addRuns(int runs)` method, which validates the input first — just as the scorer officially records runs only after the umpire signals, never directly. The `getScorecard()` public method is like the official scoreboard: it reveals what the team wants the world to know, not the internal strategy. Encapsulation is what makes large codebases maintainable: you can completely rewrite how runs are stored internally, switching from an `int` to a `RunRecord` object, without breaking any code that uses the scoreboard, just as the BCCI can change the scoring system internally without affecting how spectators read the scoreboard.

Inheritance

Inheritance allows a class to extend another class, inheriting its fields and methods while adding new behaviour or overriding existing behaviour. Java supports single class inheritance — a class extends exactly one superclass — but allows multiple interface implementation. The subclass expresses an IS-A relationship with its superclass: a `Batsman` IS-A `CricketPlayer`. The parent class defines shared behaviour, so all players have a name, a team, and a `play()` method, while subclasses specialise that behaviour for their role. The `@Override` annotation lets the compiler verify that you are correctly overriding a parent method rather than accidentally defining a new one with a mismatched signature. Inheritance promotes code reuse and models real-world hierarchies naturally, but deep inheritance chains beyond two or three levels become hard to reason about and maintain.

In cricket, every player on the field IS a CricketPlayer — they wear whites, follow the Laws, and take fielding positions. But within that shared identity, a Batsman specialises in building innings while a Bowler specialises in taking wickets. Rohit Sharma IS-A CricketPlayer who specialises as a Batsman, Jasprit Bumrah IS-A CricketPlayer who specialises as a Bowler, and Ben Stokes IS-A CricketPlayer who specialises as an AllRounder, inheriting both batting and bowling capabilities. The CricketPlayer superclass defines the shared contract: every player has a name, a team, and a play() method. The Batsman subclass inherits all of that and adds battingAverage and totalRuns fields, overriding play() to describe a batting innings. The Bowler subclass inherits the same base and overrides play() to describe a bowling spell — it does not need to re-implement "has a name" or "belongs to a team" because that comes from the parent. Java's single-inheritance rule mirrors cricket's primary role: a player's base identity is CricketPlayer, and they have ONE primary specialisation, even if an AllRounder can do both. Inheritance shows you that code reuse in OOP isn't about copy-pasting — it's about modelling the real IS-A relationship precisely, which is why it works only when the subclass genuinely is a more specific version of the parent.

Polymorphism

Polymorphism means "many forms" — the same method call produces different behaviour depending on the actual runtime type of the object. In Java this is achieved through method overriding: CricketPlayer declares play(), Batsman overrides play() with batting logic, and Bowler overrides play() with bowling logic. When you call player.play() on each element of a List<CricketPlayer>, the JVM dispatches to the correct subclass implementation at runtime using dynamic dispatch through the virtual method table. You write one loop and the JVM handles the routing to the right method for each object. Polymorphism eliminates giant if/else or switch chains based on type, making code open for extension without modification, which is the essence of the Open/Closed Principle: new types can be added without touching existing calling code.

Imagine the cricket team captain giving a single instruction to the whole playing 11: "Everyone, go play your role." Rohit Sharma the Batsman walks to the crease and builds an innings. Jasprit Bumrah the Bowler takes the ball and bowls a yorker. Ravindra Jadeja the AllRounder does whichever the match situation demands. The captain gave one instruction — "play your role" — but each player executed it completely differently. This is polymorphism exactly. The captain is your Java code: it calls player.play() on every element in List<CricketPlayer>. The JVM is the dressing room manager who knows each player's actual role (their runtime type) and routes the "play" call to the right implementation, Batsman.play() for Rohit and Bowler.play() for Bumrah. The captain doesn't need a conditional like "if Rohit, do batting stuff; if Bumrah, do bowling stuff" — the routing is automatic via the virtual method table, just as the captain doesn't individually instruct each player how to do their job. Adding a WicketKeeper to the squad doesn't require the captain to change any instructions; it just needs a new subclass with its own play() override. Polymorphism is the mechanism that lets you write code against abstractions like CricketPlayer rather than concrete types like Batsman or Bowler, which is why adding new player types never requires changing existing code.

Abstraction

Abstraction hides implementation complexity behind a simple interface, exposing only what callers actually need. In Java, abstraction is achieved through abstract classes, which cannot be instantiated and which define abstract methods that subclasses must implement, and through interfaces, which are pure contracts of method signatures with no implementation in pre-Java 8 form. `CricketPlayer` is abstract because you never instantiate a generic "player" — only concrete `Batsman` or `Bowler` objects make sense. The interface `Captain` defines the captaincy contract, with methods like `declareCaptain()` and `setFieldingPositions()`, separately from the player hierarchy, so any player type can also be a captain. Abstraction lets you reason about a system at a high level, in terms of what it does, without needing to know the implementation details of how it does it.

When the BCCI schedules an international match, they send both teams a "Playing Conditions" document. This document specifies what must happen — the match starts at 10am, each team has 11 players, the captain tosses a coin — but it says nothing about HOW Rohit Sharma should play his shots or HOW Bumrah should set his field. The Playing Conditions are the contract; each team works out the implementation details themselves. An abstract class in Java is the Playing Conditions document. `CricketPlayer` declares abstract void `play()` — it specifies that every player MUST implement `play()` but provides no implementation itself, just like Playing Conditions mandate the toss without specifying which side wins. A Java interface like `Captain` is an even purer contract, just method signatures with no implementation at all, like an ICC rule that says "the captain SHALL declare fielding positions" without dictating which positions. The abstract modifier on `CricketPlayer` prevents instantiation, because you can't field a generic "player", just as you can't play a match with "a team" — you need specifically India or Australia. Abstraction is why large systems are navigable: you can understand what a system does (the interface) without knowing how it does it (the implementation), which is the same reason coaches can plan tactics using "a fast bowler" as a concept without needing to know which specific bowler will be selected on match day.

java

```
import java.util.ArrayList;
import java.util.List;

// Abstraction: abstract class cannot be instantiated; defines a contract for all players.
abstract class CricketPlayer {
    // Encapsulation: private fields, accessible only via methods.
    private String name;
    private String team;

    public CricketPlayer(String name, String team) {
        this.name = name;
        this.team = team;
    }

    // Encapsulation: public getters expose controlled, read-only access.
    public String getName() { return name; }
    public String getTeam() { return team; }

    // Abstraction: abstract methods force every subclass to provide an implementation.
    public abstract String getRole();
    public abstract void play();
}

// Inheritance: Batsman extends CricketPlayer, reusing name/team and the contract.
class Batsman extends CricketPlayer {
    // Encapsulation: specialised private state.
    private double battingAverage;
    private int totalRuns;

    public Batsman(String name, String team, double battingAverage, int totalRuns) {
        super(name, team);
        this.battingAverage = battingAverage;
        this.totalRuns = totalRuns;
    }

    @Override // Polymorphism: overriding the parent contract.
    public String getRole() { return "Batsman"; }

    @Override // Polymorphism: JVM dispatches here at runtime for Batsman objects.
    public void play() {
```

```

        System.out.println("Batsman batting: " + getName() + " avg " + battingAverage);
    }
}

// Inheritance: Bowler extends CricketPlayer.
class Bowler extends CricketPlayer {
    // Encapsulation: specialised private state.
    private int wickets;
    private double economy;

    public Bowler(String name, String team, int wickets, double economy) {
        super(name, team);
        this.wickets = wickets;
        this.economy = economy;
    }

    @Override // Polymorphism: overriding the parent contract.
    public String getRole() { return "Bowler"; }

    @Override // Polymorphism: JVM dispatches here at runtime for Bowler objects.
    public void play() {
        System.out.println("Bowler bowling: " + getName() + " econ " + economy);
    }
}

// Abstraction: interface is a pure contract; any player type can implement it.
interface Captain {
    void declareCaptain();
    default void setFieldingPositions() {
        System.out.println("Setting field");
    }
}

// Inheritance (extends Batsman) + multiple contracts (implements Captain).
class AllRounder extends Batsman implements Captain {
    private int wickets;
    private double economy;

    public AllRounder(String name, String team, double battingAverage,
        int totalRuns, double economy) {
        super(name, team, battingAverage, totalRuns);
        this.economy = economy;
    }
}

```

```

    }

    @Override // Polymorphism: AllRounder's own role.
    public String getRole() { return "AllRounder"; }

    @Override
    public void declareCaptain() {
        System.out.println(getName() + " is the captain");
    }
}

public class Main {
    public static void main(String[] args) {
        // Polymorphism: one list holds different concrete subclasses.
        List<CricketPlayer> playing11 = new ArrayList<>();
        playing11.add(new Batsman("Rohit Sharma", "India", 48.5, 9847));
        playing11.add(new Bowler("Jasprit Bumrah", "India", 297, 2.74));
        playing11.add(new AllRounder("Ravindra Jadeja", "India", 35.2, 280, 4.1));

        // Polymorphism: JVM dispatches to correct subclass at runtime.
        playing11.forEach(p → {
            System.out.println(p.getName() + " [" + p.getRole() + "]");
            p.play();
        });
    }
}

```

Walking through the code, all four concepts appear together. Encapsulation is visible in CricketPlayer's private String name and private String team, reachable only via getName() and getTeam(); Batsman and Bowler add their own private fields the same way. Abstraction appears in the abstract class CricketPlayer and its abstract methods getRole() and play(), plus the Captain interface. CricketPlayer is abstract — not a regular class — because a generic "player" is never a valid object; only a concrete Batsman, Bowler, or AllRounder exists, and marking it abstract makes the compiler forbid new CricketPlayer(...). Inheritance appears in extends, where Batsman and Bowler reuse the parent's name/team plumbing. Polymorphism resolves p.play() inside forEach: the reference type is CricketPlayer, but the JVM looks up the actual heap object's vtable, so Rohit runs Batsman.play() and Bumrah runs Bowler.play(). AllRounder extends Batsman rather than CricketPlayer directly because an all-rounder genuinely IS-A batsman with extra bowling state, letting it reuse battingAverage and totalRuns while additionally implementing the Captain contract.

How It Works Under the Hood

Under the hood, the JVM combines lazy class loading, tiered compilation, and managed memory. Class loading is lazy: a class is loaded, verified, and initialised only on first active use, not at startup, so the ClassLoader pulls bytecode on demand. Once loaded, methods first run through the bytecode interpreter, which is immediately available but unoptimised. The HotSpot JIT then watches invocation and loop-back-edge counters: the C1 compiler kicks in for methods that warm up (roughly after about 1,000 invocations) to produce quick, lightly optimised native code, and the C2 compiler recompiles genuinely hot methods (roughly after about 10,000 invocations) with aggressive optimisations like inlining and dead-code elimination. Memory is split into the heap and per-thread stacks: each method call pushes a stack frame holding locals and operands, while objects live on the heap, with new objects in the young generation and survivors promoted to the old generation. Each class carries a virtual method table (vtable) that maps method signatures to concrete implementations, enabling O(1) polymorphic dispatch. Collectors such as G1GC reclaim unreachable objects using concurrent marking to minimise pause times.

Consider how an IPL team analyst works during a season. At the season start, the analyst doesn't study every opposition player in full depth — they load the profile of whoever is playing TODAY, which is lazy class loading. For the first few matches, the analyst uses quick generic assessments of batsmen, which is interpreter mode that works for everyone but isn't optimised. But once Virat Kohli has faced 30-plus balls across multiple matches and become "hot", the analyst produces a deep, specific breakdown of his weakness outside off-stump and his preference for the on-side against pace, and the bowler executes a targeted plan — that's JIT C2 compilation to native code, inlining the hot method. The ClassLoader is the analyst who only pulls profiles of players actually on today's playing 11, not all 700 IPL-contracted players. The JIT's C1 compiler is the quick pre-match briefing: compile fast, good enough. The C2 compiler is the multi-day analysis camp for key match-ups: compile slowly but optimise aggressively, inlining virtual calls and eliminating dead code. The vtable is the batting order — the fielding captain (calling code) checks the batting order once (vtable lookup) to know who's at the crease (which play() to call), then doesn't re-check for every delivery, giving O(1) dispatch. The garbage collector is the kit manager who recycles retired players' gear (unreachable objects) during the lunch break (concurrent GC) so it doesn't interrupt live play. The JVM's lazy loading plus tiered JIT means Java apps start fast and get faster over time as the runtime learns which code is hot — the same reason IPL teams improve their game plans as the season progresses and data accumulates.

java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

// Generics: ScoreCard works for any CricketPlayer subtype via a bounded type parameter.
class ScoreCard<T extends CricketPlayer> {
    private final List<T> players = new ArrayList<>();

    public void addPlayer(T p) {
        players.add(p);
    }

    // Streams: find the player whose name sorts last as a simple top-performer stand-in.
    public T getTopPerformer() {
        return players.stream()
            .max(Comparator.comparing(CricketPlayer::getName))
            .orElse(null);
    }

    public void displayAll() {
        players.forEach(p →
            System.out.println(p.getRole() + ": " + p.getName()));
    }
}

class ScoreCardDemo {
    public static void main(String[] args) {
        ScoreCard<Batsman> batsmen = new ScoreCard<>();
        batsmen.addPlayer(new Batsman("Rohit Sharma", "India", 48.5, 9847));
        batsmen.addPlayer(new Batsman("Virat Kohli", "India", 53.6, 13848));
        batsmen.displayAll();

        ScoreCard<Bowler> bowlers = new ScoreCard<>();
        bowlers.addPlayer(new Bowler("Jasprit Bumrah", "India", 297, 2.74));
        System.out.println("Top: " + bowlers.getTopPerformer());
    }
}
```

💡 Pro Tip: Use records for immutable value objects like `BallDelivery(int over, int ball, int runs, boolean isWicket)` — Java 16+ records auto-generate `equals/hashCode/toString` with zero boilerplate. Prefer interfaces over abstract classes when you don't have shared state to inherit, because interfaces support multiple implementation (a Batsman can also be Captain) while abstract class inheritance is single.

Common Patterns & Best Practices

Three patterns recur constantly in well-structured Java. The Builder pattern handles complex object construction: a `Match` needs `team1`, `team2`, `venue`, `format`, and `overs`, and expressing every combination through telescoping constructors quickly becomes unmanageable and error-prone. A Builder instead provides a fluent API where each setter returns the builder, and the final `build()` method centralises validation so an invalid `Match` object can never be created. The Factory pattern decouples creation from usage: a call like `PlayerFactory.create("batsman", stats)` hides which concrete subclass is instantiated, so callers depend only on the abstract type. The Strategy pattern encapsulates interchangeable algorithms behind a common interface: a `Match` holds a `ScoringStrategy`, and concrete strategies such as `T20Strategy`, `TestStrategy`, and `ODIStrategy` can be swapped in to change scoring rules without modifying the `Match` class itself, keeping format logic isolated and independently testable.

Before an ICC match begins, the Match Referee completes a Pre-Match Inspection Checklist, confirming the pitch report, floodlight certification, DRS camera availability, ball brand, playing hours, and powerplay overs allocation. Each item is confirmed one by one with the ground authority. Only after every item is ticked does the referee issue the official "Match Approved" certificate, and if the DRS cameras aren't certified the referee won't approve, with no workarounds allowed. The Builder pattern is this checklist process. Each builder method like `withTeam1("India")`, `withVenue("Wankhede")`, and `withFormat(Format.T20)` ticks one condition. The `build()` method is the referee's final review: it checks that all required fields are set and throws `IllegalStateException`, meaning "match cannot proceed", if any are missing, which prevents an invalid `Match` object from ever existing. Without a Builder, calling `Match(team1, team2, null, null, 0)` is like starting a match without confirming the venue, because the system allows invalid state that causes failures later. The Strategy pattern maps to cricket formats: the same `Match` class (the ground) swaps in `T20ScoringStrategy` (6-over powerplay, super over tiebreak), `ODIScoringStrategy` (50 overs, DLS method), or `TestScoringStrategy` (no over cap, follow-on rule), so the ground doesn't change, only the rules applied. Builder and Strategy together show the Open/Closed Principle in practice: you add new match formats (a new Strategy) or new required conditions (a new Builder field) without changing the `Match` class itself.

java

```
enum Format { T20, ODI, TEST }

// Strategy: a common contract for swappable scoring rules.
interface MatchScoringStrategy {
    int scoreBall(int runs);
    String getFormatName();
}

class T20ScoringStrategy implements MatchScoringStrategy {
    @Override public int scoreBall(int runs) { return runs; }
    @Override public String getFormatName() { return "T20"; }
}

class TestScoringStrategy implements MatchScoringStrategy {
    @Override public int scoreBall(int runs) { return runs; }
    @Override public String getFormatName() { return "Test"; }
}

class Match {
    private final String team1;
    private final String team2;
    private final String venue;
    private final Format format;
    private final int overs;

    private Match(Builder b) {
        this.team1 = b.team1;
        this.team2 = b.team2;
        this.venue = b.venue;
        this.format = b.format;
        this.overs = b.overs;
    }

    // Builder: fluent construction with validation centralised in build().
    static class Builder {
        private String team1;
        private String team2;
        private String venue;
        private Format format;
        private int overs;
    }
}
```

```

        public Builder withTeam1(String t) { this.team1 = t; return this; }
        public Builder withTeam2(String t) { this.team2 = t; return this; }
        public Builder withVenue(String v) { this.venue = v; return this; }
        public Builder withFormat(Format f) { this.format = f; return this; }
        public Builder withOvers(int o) { this.overs = o; return this; }

        public Match build() {
            if (team1 == null || team2 == null || venue == null || format == null) {
                throw new IllegalStateException("Match cannot proceed: missing required
fields");
            }
            return new Match(this);
        }
    }
}

class MatchDemo {
    public static void main(String[] args) {
        Match wc = new Match.Builder()
            .withTeam1("India")
            .withTeam2("Australia")
            .withVenue("Narendra Modi Stadium")
            .withFormat(Format.ODI)
            .withOvers(50)
            .build();

        MatchScoringStrategy strategy = new T20ScoringStrategy();
        System.out.println("Built match using " + strategy.getFormatName() + " strategy");
    }
}

```

 **Warning:** Never compare Integer objects with == — Java caches Integer from -128 to 127 only. Comparing Integer score == 150 works for small scores but silently returns false for centuries, because above 127 each Integer.valueOf(150) creates a new heap object. Always use .equals(): score.equals(150). This causes subtle production bugs in cricket scoring systems where century scores appear to not match targets.

Real-World Application

Java runs much of cricket's digital infrastructure at scale. Cricinfo serves match data through Java backend APIs, Dream11 handles 100M-plus users during the IPL on Spring Boot microservices, BCCI

official scoring systems run on the JVM, and Hotstar manages streaming metadata for tens of millions of concurrent viewers, while Android apps run Kotlin on the JVM. Java 21 virtual threads from Project Loom enable millions of simultaneous WebSocket connections for live score pushes without the overhead of one OS thread per request. These systems use exactly the patterns above: OOP models domain entities like players and matches, the Builder pattern creates valid match and player objects, and the Strategy pattern swaps format-specific scoring rules at runtime.

KEY POINTS

- Encapsulation prevents invalid object state by hiding fields behind methods that validate before modifying internal data
- Inheritance models IS-A relationships and enables code reuse, but deep chains beyond 3 levels become hard to maintain
- Polymorphism via vtable dispatch lets one method call route to multiple implementations with zero conditional logic in calling code
- Abstraction via interfaces decouples what a system does from how it does it, enabling multiple independent implementations
- Builder pattern with validation in build() makes invalid object construction physically impossible, not just conventionally discouraged
- Java 21 virtual threads (Project Loom) enable millions of concurrent connections without thread-per-request overhead, critical for live score WebSockets

Q: A `List<CricketPlayer>` holds Batsman and Bowler objects. You call `player.play()` in a `forEach` loop. Which mechanism ensures `Batsman.play()` runs for batsmen and `Bowler.play()` for bowlers?

- A** Static binding at compile time based on the `List<CricketPlayer>` declaration
- B** Dynamic dispatch via vtable lookup at runtime based on the actual heap object type
- C** The `instanceof` operator inserted by the compiler before each call
- D** The `@Override` annotation signals the JVM which method to call at runtime

Explanation: Java uses dynamic dispatch for non-static, non-private instance methods. Each class has a vtable mapping method signatures to implementations. At runtime, the JVM looks up the vtable of the ACTUAL object type (Batsman or Bowler), not the declared reference type (CricketPlayer). `@Override` only catches signature mismatches at compile time and has no effect at runtime.

Q: You're designing a cricket scoring app. CricketPlayer needs shared behaviour (name, team fields). Batsman and Bowler have completely different state. Captain is a role any player type can hold. What is the correct Java design?

- A** abstract class CricketPlayer with Batsman/Bowler subclasses; Captain as a separate interface
- B** interface CricketPlayer implemented by Batsman and Bowler; Captain extending CricketPlayer
- C** One CricketPlayer class with a role field (BATSMAN/BOWLER/CAPTAIN) and conditional methods
- D** Three unrelated classes with no inheritance — avoid coupling

Explanation: Abstract class CricketPlayer provides shared STATE (name, team fields) that interfaces cannot. Batsman and Bowler extend it, inheriting shared state while adding specialised fields. Captain as a separate interface allows any player type to also be captain (AllRounder extends Batsman implements Captain) — Java's multiple interface rule handles the 'role overlay' without forcing single-choice inheritance.